Task 1: Nginx Web Server – Installation and Hosting Configuration

Objective

Install Nginx web server on your local machine
Host a simple website[index.html] in the default document root, by using Nginx web server.
Call the website using "firstnginx.test"
Study in deep about nginx service, hosting types and default configurations in Nginx web server

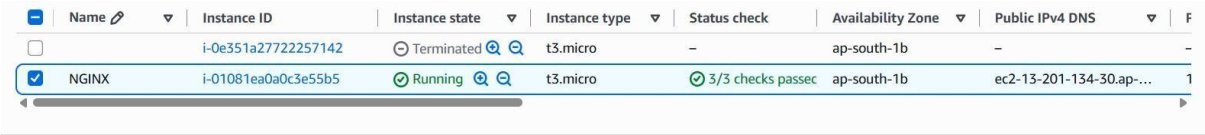
Requirements

- AWS EC2 instance running RHEL
- Nginx web server
- A simple index.html webpage
- AWS Security Group configuration
- Windows client machine for browser testing

Procedure

Step 1 – Launch the EC2 Instance

An AWS EC2 instance was launched to serve as the RHEL machine for hosting the Nginx web server.



Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	F
	i-0e351a27722257142	Terminated	t3.micro	–	ap-south-1b	–	–
NGINX	i-01081ea0a0c3e55b5	Running	t3.micro	3/3 checks passec	ap-south-1b	ec2-13-201-134-30.ap-...	1

Step 2 – Install Nginx

The Nginx web server package was installed on the RHEL EC2 instance using the dnf package manager.

```
sudo dnf install nginx -y
```

After installation, the Nginx service was enabled so that it starts automatically on system boot.

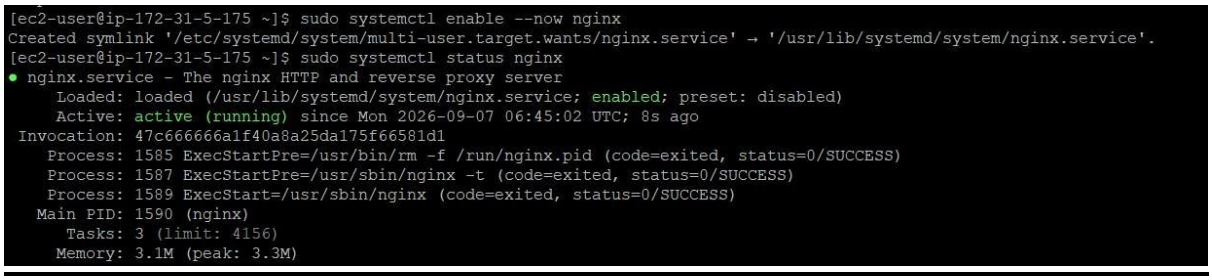
```
sudo systemctl enable –now nginx
```

The status of the service was verified using:

```
sudo systemctl status nginx
```

The Nginx service was confirmed to be active and running.





Step 3 – Identify the Default Document Root

The Nginx configuration file was checked to identify the default document root used by the web server.

```
cat /etc/nginx/nginx.conf
```

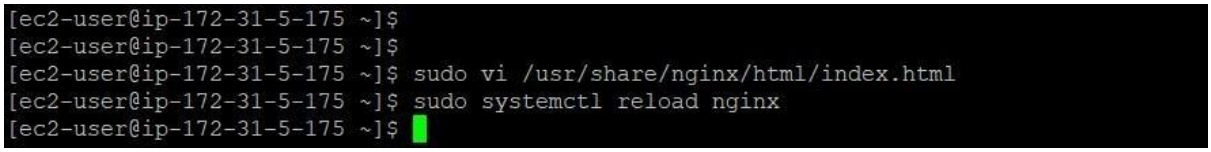
The default document root for Nginx on RHEL was identified as /usr/share/nginx/html.

The index.html file inside this default document root was then edited and replaced with simple custom webpage content.

```
sudo vi /usr/share/nginx/html/index.html
```

Nginx was reloaded to apply the change:

```
sudo systemctl reload nginx
```



Step 4 – Test Using the Public IP Address

After hosting the website in the default document root, the webpage was tested directly using curl on the EC2 instance itself:

curl http://localhost

The custom HTML content was returned successfully, confirming that Nginx was serving the updated index.html file correctly from the default document root.

The public IP address of the EC2 instance was then tested using curl to confirm that the website was also reachable externally:

curl http://<EC2-Public-IP>

The same custom webpage content was returned successfully, confirming that Nginx was correctly serving the website over the network and that the AWS Security Group was permitting inbound HTTP traffic on port 80.

Step 5 – Configure the Custom Hostname (Local DNS Override)

To access the website using the required hostname firstnginx.test, the concept of a local DNS override using the hosts file was used, since firstnginx.test is not a registered public domain.

On the Windows client machine, the hosts file was edited with administrator privileges:

`C:\Windows\System32\drivers\etc\hosts`

The following entry was added, mapping the EC2 instance's public IP address to the custom hostname:

`<EC2-Public-IP>  firstnginx.test`

This allowed the Windows machine to resolve firstnginx.test directly to the Nginx server's IP address, bypassing real DNS entirely.

```
#      102.54.94.97     rhino.acme.com          # source server
#       38.25.63.10     x.acme.com              # x client host

# localhost name resolution is handled within DNS itself.
#   127.0.0.1       localhost
#   ::1             localhost
   13.201.134.30   firstnginx.test
```

Step 6 – Verify the Custom Domain

The custom hostname was tested by entering the following address into the Windows web browser:

`http://firstnginx.test`

The custom webpage content was displayed successfully, confirming that the website was accessible using the custom hostname.



Task 2 – Custom Branded 404 Error Page in Nginx

Objective

The Marketing department required visitors to see a branded custom error page instead of the default Nginx 404 "Not Found" page. This task involved creating a custom 404 page and configuring Nginx to serve it whenever a requested page was not found.

Step 2 – Create the Custom 404 Page

A custom-branded 404.html file was created inside Nginx's default document root:

```
sudo vi /usr/share/nginx/html/404.html
```

The page was designed with a dark theme, a bold "404" heading, a short message, and a link back to the homepage.

```
[ec2-user@ip-172-31-8-110 ~]$ cat /usr/share/nginx/html/404.html
<!DOCTYPE html>
<html>
<head>
    <title>Page Not Found</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            padding: 100px;
            background-color: #1a1a2e;
            color: #ffffff;
        }
        h1 {
            font-size: 60px;
            color: #e94560;
            margin-bottom: 10px;
        }
        p {
            font-size: 20px;
        }
        a {
            color: #4fc3f7;
            text-decoration: none;
            font-weight: bold;
        }
    </style>
</head>
<body>
    <h1>404</h1>
    <p>Oops! The page you're looking for doesn't exist.</p>
    <p><a href="/">Go back home</a></p>
</body>
</html>
```

Step 3 – Configure Nginx to Serve the Custom 404 Page

The Nginx configuration file was edited to define the custom error page: sudo

vi /etc/nginx/nginx.conf

The following directives were added inside the existing server block:

error_page 404 /404.html; location = /404.html { root /usr/share/nginx/html; internal; }

```
    include             /etc/nginx/mime.types;
    default_type        application/octet-stream;

    # Load modular configuration files from the /etc/nginx/conf.d directory.
    # See http://nginx.org/en/docs/ngx_core_module.html#include
    # for more information.
    include /etc/nginx/conf.d/*.conf;

    server {
        listen       80;
        listen       [::]:80;
        server_name  _;
        root         /usr/share/nginx/html;

        # Load configuration files for the default server block.
        include /etc/nginx/default.d/*.conf;

    error_page 404 /404.html;
    location = /404.html {
        root /usr/share/nginx/html;
        internal;

    }
}
```

Step 5 – Test and Apply the Configuration

sudo nginx -t sudo
systemctl reload nginx

The configuration test returned "syntax is ok" and "test is successful."

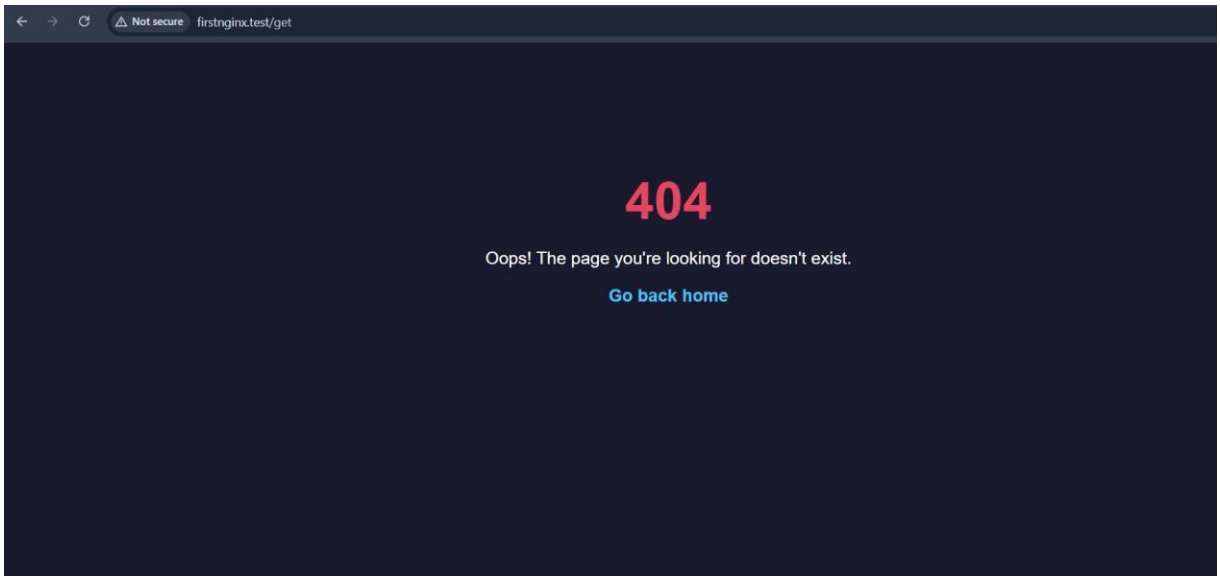
```
[ec2-user@ip-172-31-8-110 ~]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
[ec2-user@ip-172-31-8-110 ~]$ sudo systemctl reload nginx
[ec2-user@ip-172-31-8-110 ~]$
```

Step 6 – Verification

The custom 404 page was tested by requesting a non-existent page using the previously configured hostname:

http://firstnginx.test/new

The branded custom 404 page was displayed successfully instead of the default Nginx error page, confirming that the configuration was working as intended.



Verification

- The Nginx package was successfully installed on the RHEL EC2 instance.
- The Nginx service was started and enabled to run automatically on boot.
- The default document root used by Nginx was identified as /usr/share/nginx/html.
- The index.html file was modified with custom webpage content.
- The AWS Security Group was verified to allow inbound HTTP traffic on port 80.
  The website was successfully reached using the EC2 instance's public IP address.
- The custom hostname firstnginx.test was set up using a local hosts file entry on the Windows machine.
- The hosted webpage was successfully displayed in the Windows browser using firstnginx.test.
- A custom, branded 404.html page was created in the default document root.
- The error_page directive was configured to serve the custom page for 404 errors.
- A configuration syntax error was identified and correctly resolved.
- The custom 404 page was successfully displayed when a non-existent page was requested.

Conclusion

Through this task, the Nginx web server was successfully installed and configured on an AWS EC2 instance running RHEL. A simple website was hosted directly in Nginx's default document root and verified using both the instance's public IP address and the custom hostname firstnginx.test. The AWS Security Group was checked to ensure external HTTP access was permitted, and a local hosts file entry was used to map the custom hostname to the server, allowing the website to be accessed successfully from a Windows browser. In addition, a custom branded 404 error page was created and configured using Nginx's error_page directive, replacing the default error page as requested by the Marketing department, with a configuration syntax error successfully identified and resolved along the way. This task provided practical, hands-on experience with Nginx installation, website hosting, cloud networking basics, local domain resolution techniques, and custom error page configuration.